

Central de Solicitações ao Marketing — Documento para Desenvolvimento

O que falta para transformar o protótipo navegável em uma plataforma real, multiusuário e pronta para produção.

Documento para o programador responsável · VendaMais

Resumo em uma frase: já existe um protótipo web completo, com todas as telas e toda a lógica de cálculo funcionando. O que falta é a camada de servidor (banco de dados central, login seguro e envio de e-mail) para que vários usuários compartilhem os mesmos dados sem perdê-los.

1. Situação atual

Existe um protótipo funcional e navegável (um único aplicativo web) que já contém:

- Login por perfil (solicitante, administrador, gestor/diretoria);
- Página inicial do solicitante, formulário de nova solicitação com **cálculo de viabilidade em tempo real**, "minhas solicitações", fila geral resumida;
- Painel administrativo, calendário de produção, configuração de tipos/pesos/prazos e de capacidade semanal;
- Simulação de e-mails automáticos (caixa de saída na tela).

Toda a lógica de negócio já está implementada e validada — cálculo de peso, prazo mínimo em dias úteis, ocupação e capacidade semanal, classificação de viabilidade (verde/amarelo/vermelho), busca da próxima semana disponível e pontuação de prioridade.

Limitação atual: é apenas front-end. Os dados são gravados no navegador de cada pessoa (localStorage). Isso significa que os dados **não são compartilhados entre usuários nem entre dispositivos**, podem se perder ao limpar o navegador, e as senhas da equipe estão no código (apenas para demonstração). Serve para validar o fluxo — não para uso real multiusuário.

2. O que precisa ser construído

1. **Banco de dados central** — armazenar usuários, solicitações, tipos de demanda, capacidades, histórico, comentários e logs de e-mail de forma permanente e compartilhada.

2. **Autenticação e controle de acesso** — login seguro com senha criptografada; cada perfil enxerga apenas o que lhe é permitido.
3. **API** — camada que conecta as telas ao banco (criar/ler/atualizar solicitações, configurações, etc.).
4. **Envio de e-mails reais** — disparar de verdade os 6 e-mails automáticos que hoje são apenas simulados.
5. **Deploy e ambiente** — hospedagem com HTTPS e variáveis de ambiente para as chaves.
6. **Reaproveitar a lógica existente** — portar para o servidor (ou manter no front consumindo a API) os cálculos já prontos, sem reescrever as regras.

3. Stack sugerida (flexível)

Sugestão que se integra bem com o que já existe e tem plano gratuito para começar. O desenvolvedor pode usar equivalentes — o importante são os requisitos das seções seguintes.

Necessidade	Sugestão	Alternativas
Banco de dados + Autenticação	Supabase (Postgres + Auth)	Firebase, Postgres/Neon
Hospedagem / Front-end	Vercel	Netlify, Cloudflare Pages
API / Backend	Next.js (API routes) ou funções do Supabase	Node/Express
Envio de e-mail	Resend	SendGrid, SMTP corporativo
Upload de arquivos	Supabase Storage	S3, link externo (Drive)

4. Modelo de dados

As tabelas já foram desenhadas na especificação original e estão refletidas no protótipo.
Estrutura mínima:

Tabela	Função
users	Usuários e perfis (solicitante, admin, gestor)
request_types	Tipos de demanda, peso padrão e prazo mínimo
weekly_capacities	Capacidade (pontos) por semana e bloqueios
requests	Solicitações (tabela principal, com briefing e campos internos)
request_files	Anexos e links de cada solicitação
request_comments	Comentários internos e ao solicitante
status_history	Histórico de mudanças de status e ações
effort_allocations	Distribuição de esforço de demandas grandes entre semanas
holidays	Feriados e dias bloqueados (para o cálculo de dias úteis)
email_logs	Registro dos e-mails enviados

5. Regras de negócio que não podem se perder

Esta é a parte mais sensível do sistema. Toda a lógica abaixo **já existe no protótipo** e pode ser portada — recomenda-se solicitar o código-fonte ao responsável e reaproveitar, em vez de reescrever do zero.

- **Peso** da demanda definido pelo tipo (configurável pelo admin).
- **Prazo mínimo** calculado em dias úteis (pula fins de semana e feriados cadastrados).
- **Semana** considerada de segunda a sexta.
- **Ocupação da semana** = soma dos pesos das demandas com entrega prevista na semana, contando apenas os status: *Aprovado para produção, Em produção, Em revisão, Ajustes solicitados* (mais demandas marcadas como "reservar capacidade").
- **Capacidade disponível** = capacidade configurada da semana – ocupação.
- **Viabilidade**: verde (cabe no prazo e na capacidade), amarelo (no limite / depende de aprovação / $\geq 90\%$ da capacidade), vermelho (prazo abaixo do mínimo, semana sem capacidade, briefing incompleto ou anexo obrigatório ausente).
- **Próxima semana disponível**: quando a semana pedida está cheia, avançar semana a semana até encontrar capacidade suficiente e sugerir a data.
- **Pontuação de prioridade** = prioridade base + impacto comercial + proximidade da data de uso – penalidades por briefing incompleto.

6. Autenticação e permissões

Perfil	Acesso	Permissões
Solicitante	Cadastro por e-mail (sem senha ou via link mágico)	Abrir solicitações; ver apenas as próprias ; ver fila geral resumida
Administrador	E-mail + senha	Acesso total: triagem, edição, configurações, capacidade
Gestor / Diretoria	E-mail + senha	Somente leitura: fila, calendário, indicadores

⚠ Remover as senhas de demonstração do código. Senhas devem ficar no banco com hash (a maioria das soluções de Auth já faz isso). Demandas confidenciais só aparecem para o administrador e para o próprio solicitante; comentários internos só para administradores.

7. E-mails automáticos a conectar

Os textos já existem no protótipo. Basta ligá-los a um serviço real de envio:

- Solicitação recebida (para o solicitante);
- Nova solicitação (para o marketing);
- Briefing incompleto / devolução;
- Prazo aprovado;
- Prazo reprogramado;
- Solicitação entregue.

8. Critérios de aceite

A versão em produção estará pronta quando:

- Vários usuários, em dispositivos diferentes, acessam e veem **os mesmos dados**;
- O administrador vê todas as solicitações; o solicitante vê apenas as suas;
- Nenhum dado se perde ao fechar ou limpar o navegador;
- Os cálculos de peso, prazo, capacidade e viabilidade funcionam igual ao protótipo;
- Login com senha funciona e é seguro;
- Os e-mails automáticos são realmente enviados;
- O admin consegue configurar tipos, pesos, prazos e capacidade semanal, e isso reflete nos cálculos.

9. Observações importantes

- **Começar com a base zerada**, sem os dados de teste do protótipo — mas já com os **24 tipos de demanda** e as **capacidades semanais padrão** pré-carregados.
- A partir do go-live, **tudo o que for cadastrado fica salvo no banco central e não se perde** (diferente do protótipo atual).
- Segurança: HTTPS, senhas com hash, chaves em variáveis de ambiente.
- LGPD: são dados de usuários internos; manter o acesso restrito conforme os perfis.
- O protótipo continua útil como referência visual e de comportamento — o desenvolvedor deve reproduzir as mesmas telas e o mesmo fluxo.

Qualquer dúvida sobre as regras de cálculo ou o comportamento das telas, consultar o protótipo navegável e a especificação original do projeto, que detalham cada caso (incluindo as mensagens automáticas e os exemplos numéricos de capacidade).